

Mini-progetto: *Creature Loader Web* (MVT · Flask · SQLite)

Contesto

Nel progetto precedente hai sviluppato un **Creature Loader** che:

- recupera dati da una API HTTP,
- normalizza i dati in un dizionario,
- crea un oggetto *Creature*,
- salva e legge i dati.

In questo esercizio devi **portare la stessa logica in un'applicazione web**, utilizzando **Flask** e **SQLite**, seguendo un'architettura **MVT (Model · View · Template)**.

Obiettivo

Realizzare una piccola applicazione Flask che permetta di:

1. visualizzare l'elenco dei Pokémon salvati nel database,
 2. aggiungere un nuovo Pokémon inserendo solo il nome,
 3. scaricare automaticamente i dati dalla API,
 4. salvare i dati nel database SQLite,
 5. modificare i dati di un Pokémon già esistente.
-

◆ Stack richiesto

- Python
- Flask

- SQLite (tramite SQLAlchemy Core/ORM)
- HTML (template Jinja2)
- `requests`

- ⚠ Non è richiesta autenticazione
 - ⚠ Non è richiesto CSS avanzato
 - ⚠ Non è richiesta validazione lato client
-

◆ Architettura MVT (semplificata)

Model → Creature (e database SQLite)
View → route Flask (controller logico)
Template → HTML (Jinja2)

Separare chiaramente:

- logica HTTP/API
 - logica database
 - rendering HTML
-

🧱 Modello dati (Model)

Nel database deve essere presente una tabella `creatures` con almeno i seguenti campi:

- `id` (primary key, int)
- `name` (text)
- `height` (int)
- `weight` (int)
- `types` (text)

- `stats` (text)

⚠ `types` e `stats` possono essere salvati come **JSON serializzato in stringa**

Creature (Python)

La classe `Creature` deve:

- accettare i dati nel costruttore tramite `**dict`
- poter essere inizializzata sia:
 - dai dati scaricati dalla API
 - dai dati letti dal database

Esempio concettuale:

```
creature = Creature(**data_dict)
```

◆ Reader HTTP

Riutilizzare la logica del **Creature Loader precedente**:

- richiesta HTTP alla PokeAPI
- normalizzazione dei dati
- gestione degli errori con `try / except`

Il Reader **non deve**:

- scrivere direttamente nel database
 - restituire HTML
-



Template richiesti

1 `index.html`

Pagina principale dell'applicazione.

Deve mostrare:

- elenco di tutti i Pokémon presenti nel database
- Una scheda (riquadro) per ciascun Pokémon con:
 - nome e stats principali
 - link per la modifica

Inoltre deve contenere:

- un pulsante o link **“Aggiungi Pokémon”**
-

2 `add.html`

Pagina con un form contenente:

- un solo campo input: `nome_pokemon`
- un pulsante di invio

Comportamento:

- alla sottomissione del form:
 1. viene fatta la richiesta HTTP alla API
 2. i dati vengono salvati nel database
 3. si viene reindirizzati alla pagina `index`
-

3 `edit.html`

Pagina per modificare un Pokémon già presente.

Deve permettere:

- visualizzazione dei dati attuali
- modifica di uno o più campi (es. peso, altezza)

Alla conferma:

- i dati vengono aggiornati nel database
 - redirect alla pagina [index](#)
-

◆ Route Flask richieste

Almeno le seguenti:

- [/](#)
→ mostra l'elenco dei Pokémon (index)
 - [/add](#)
→ GET: mostra il form
→ POST: scarica i dati e salva nel DB
 - [/edit/<id>](#)
→ GET: mostra il form di modifica
→ POST: aggiorna il record nel DB
-



Gestione delle eccezioni (obbligatoria)

Devono essere gestiti:

- Pokémon non trovato nella API
- errori di rete

L'app **non deve crashare**:

mostrare un messaggio di errore o tornare alla pagina precedente.



Consegna richiesta

- progetto Flask funzionante
 - file `.py` principali
 - cartella `templates/` con i file HTML
 - file SQLite con **almeno 3 Pokémon salvati**
-



Criteri di valutazione

Aspetto	Valutazione
Architettura MVT	
Uso corretto di Flask	
Integrazione API esterna	
Persistenza su SQLite	
Uso corretto di template HTML	
Gestione degli errori	

Note finali

- L'obiettivo **non è l'estetica**, ma la **logica**
- Il progetto deve dimostrare la continuità con il Creature Loader precedente
- Codice chiaro > codice complesso